

Planejamento de Rotas para Transporte Estudantil

Estrutura de Dados

Apresentação de Projeto

Estudantes: Benjamin da Conceição Neves Cardoso; Caio Conceição dos Santos; Daniel Bezerra Medeiros de Souza; Daniel José Cerqueira Brito; Jessé dos Santos Nery; João Pedro Carneiro da Silva;

Professores: Dr. Tiago Palma Pagano

Cursos: Bacharelado em Ciências Exatas e Tecnológicas e Bacharelado em Engenharia de Computação

18 de Julho de 2026

Resumo

A mobilidade dos estudantes em Cruz das Almas-BA envolve o desafio de conectar de forma eficiente diversos bairros e pontos de embarque às escolas e instituições de ensino da cidade. Muitas vezes, quando os trajetos não são bem definidos, as viagens acabam se tornando mais longas, gerando eventuais atrasos e maiores gastos com combustível. Pensando em tornar esse processo mais prático, este projeto apresenta um sistema interativo desenvolvido em Java que mapeia a cidade e utiliza técnicas de busca e rota para calcular e sugerir os melhores caminhos, ajudando a planejar um transporte estudantil mais ágil e organizado.

Palavras-chave: transporte estudantil; planejamento de rotas; teoria dos grafos; Java.

Contextualização do tema

- A mobilidade urbana é um desafio constante; logo, o planejamento de rotas para o transporte em Cruz das Almas-BA é essencial para o acesso dos alunos.
- Este problema real de logística e transporte pode ser modelado matematicamente por meio da Teoria dos Grafos [1].
- No sistema (desenvolvido em Java/Swing), bairros, pontos, escolas e universidades são vértices, enquanto as ruas são arestas ponderadas pela distância.
- A proposta aplica algoritmos clássicos de roteirização para tornar o transporte estudantil visual, interativo e mais eficiente [1].

Problematização

- O estudo de grafos exige relacionar estruturas abstratas, como vértices, arestas e pesos, a situações concretas de deslocamento urbano.
- Embora algoritmos de grafos determinem a sequência de vértices, ainda é necessário transformar essa solução em um percurso contínuo sobre ruas reais, permitindo visualizar distâncias, duração e deslocamento do veículo.
- O problema consiste em integrar algoritmos de grafos, coordenadas geográficas, serviços de roteamento e visualização cartográfica em uma única aplicação desktop.
- A solução também deve permanecer utilizável quando o serviço externo de rotas estiver indisponível, recorrendo a cálculos e geometria locais.
- **Questão norteadora:** Como desenvolver, somente com Java e bibliotecas locais, um sistema que calcule, represente e simule rotas contínuas em um mapa real de Cruz das Almas?

Justificativa

- **Impacto Social e Logístico:** A otimização das rotas reduz o tempo de viagem dos alunos (melhorando a qualidade de vida) e otimiza os recursos da gestão de transporte.
- **Relevância Acadêmica:** Materializa a teoria vista em sala de aula, demonstrando a utilidade real de Estruturas de Dados e Teoria dos Grafos em problemas do cotidiano.
- **Aplicação Prática:** A criação de uma interface gráfica (GUI) interativa que automatiza um trabalho que seria custoso se feito de forma manual.
- **Diferencial em relação aos trabalhos relacionados:** Estudos anteriores concentram-se na formulação e na otimização matemática do roteamento de ônibus escolares [2, 3]. Este projeto integra, em uma aplicação didática voltada a Cruz das Almas, diferentes algoritmos de grafos, mapa real, simulação visual, exportação de resultados e fallback local.

Objetivos

Objetivo geral

Desenvolver um sistema desktop capaz de calcular, visualizar e simular rotas para o transporte estudantil utilizando Teoria dos Grafos e algoritmos clássicos de roteamento.

Objetivos específicos

- Modelar o mapa urbano de Cruz das Almas como um grafo com pontos, conexões e pesos.
- Implementar Dijkstra, Prim/Kruskal e roteirização por Caixeiro viajante simplificado.
- Priorizar bairros e pontos com maior número de estudantes.
- Buscar reduzir a distância percorrida, o tempo de viagem e o consumo de combustível e atrasos.
- Construir uma interface gráfica para exibir mapa, cálculos e resultados em tempo real.

Grafos

- Grafos são estruturas compostas por vértices e arestas, representadas no sistema por lista de adjacência [1].
- Vértices: UFRB, bairros, pontos de embarque e áreas de coleta de estudantes.
- Arestas: ruas e trajetos entre pontos do mapa.
- Peso das arestas: distância em quilômetros entre os pontos conectados.
- A fórmula de Haversine calcula a distância geográfica entre coordenadas de latitude e longitude.

Algoritmos: Dijkstra

- Dijkstra calcula o menor caminho entre dois pontos do grafo usando estratégia gulosa e fila de prioridade [1].

Código 1: Pseudocódigo do algoritmo de Dijkstra.

```
1 FUNCAO Dijkstra(grafo, origem):  
2   PARA cada ponto p do grafo:  
3     distancia[p] = INFINITO  
4   distancia[origem] = 0  
5   fila = FilaPrioridade ordenada por distancia  
6   fila.inserir(origem)  
7   ENQUANTO fila nao vazia:  
8     atual = fila.remover_menor()  
9     PARA cada (vizinho, peso) em vizinhos(atual):  
10      nova = distancia[atual] + peso  
11      SE nova < distancia[vizinho]:  
12        distancia[vizinho] = nova  
13        anterior[vizinho] = atual  
14        fila.inserir(vizinho)  
15   RETORNAR distancia, anterior
```


Algoritmos: Prim

- Prim gera a Árvore Geradora Mínima (MST), conectando todos os pontos com menor custo total [1].

Código 2: Pseudocódigo do algoritmo de Prim.

```
1 FUNCAO Prim(grafo, inicio):  
2   visitados = {inicio}  
3   mst = lista vazia  
4   fila = FilaPrioridade de arestas por distancia  
5   fila.inserir(arestas de inicio)  
6  
7   ENQUANTO fila nao vazia:  
8     aresta = fila.remover_menor()  
9     SE aresta.destino NAO esta em visitados:  
10      visitados.adicionar(aresta.destino)  
11      mst.adicionar(aresta)  
12      fila.inserir(arestas de aresta.destino)  
13  
14   RETORNAR mst
```

Algoritmos: Kruskal, BFS, DFS e Guloso

- **Kruskal:** gera uma Árvore Geradora Mínima ao escolher as arestas de menor peso sem formar ciclos [1].
- **BFS (Busca em Largura):** percorre o grafo por níveis utilizando uma fila [1].
- **DFS (Busca em Profundidade):** explora cada ramo antes de retroceder, utilizando recursão ou pilha [1].
- **Guloso por demanda:** prioriza, a cada etapa, o ponto não visitado com maior número de estudantes; é rápido, mas não garante a melhor rota global.

Algoritmos: Caixeiro viajante simplificado

- Caixeiro viajante simplificado usa a heurística do vizinho mais próximo para definir uma sequência eficiente de paradas [1].

Código 3: Pseudocódigo da heurística Caixeiro viajante simplificado pelo vizinho mais próximo.

```
1 FUNCAO RotaOtimizada(grafo, origem):  
2     visitados = {origem}  
3     percurso = [origem]  
4     atual = origem  
5  
6     ENQUANTO existir ponto nao visitado:  
7         distancias = Dijkstra(grafo, atual)  
8         proximo = ponto nao visitado com menor distancia  
9         percurso.adicionar(caminho ate proximo)  
10        visitados.adicionar(proximo)  
11        atual = proximo  
12  
13    RETORNAR percurso, distancia_total, tempo_total
```

Desenvolvimento

- Pesquisa aplicada, de natureza exploratória, desenvolvida por meio da implementação de um protótipo computacional.
- Aplicação desktop implementada em Java Swing, utilizando apenas o JDK e arquivos JAR armazenados localmente em `lib/`.
- Interface cartográfica construída com JXMapView e tiles do OpenStreetMap.
- Mapa iniciado em Cruz das Almas ($-12,6723$; $-39,1054$), com 14 bairros e 18 escolas georreferenciados em WGS84.
- Os pontos foram classificados como bairro, escola, universidade ou ponto de embarque e representados por marcadores distintos.

Modelagem e estruturas de dados

- A malha de pontos foi modelada como um grafo ponderado, armazenado por lista de adjacência.
- Cada vértice reúne identificação, tipo, latitude, longitude e, quando aplicável, demanda de estudantes.
- Cada aresta registra a ligação entre dois pontos e o respectivo peso de distância.
- Operações de cadastro, edição e remoção permitem alterar o grafo; mudanças de coordenadas atualizam a geometria e as distâncias relacionadas.
- Filtros por tipo de ponto e estatísticas auxiliam a inspeção dos dados cadastrados.

Processamento dos algoritmos

- O usuário seleciona origem, destino e, conforme o algoritmo, os demais pontos que participarão do cálculo.
- Foram disponibilizados Dijkstra, Prim, Kruskal, BFS, DFS, estratégia gulosa por demanda e TSP simplificado.
- Cada algoritmo determina uma ordem de visita ou um subconjunto de arestas, conforme sua finalidade.
- Transições entre vértices não adjacentes são expandidas em caminhos contínuos, evitando saltos no desenho e na animação.

Algoritmos Implementados

Tabela 1: Algoritmos implementados e suas respectivas finalidades no sistema.

Algoritmo	Finalidade
Dijkstra	Cálculo do menor caminho entre dois pontos do grafo.
Prim	Construção da árvore geradora mínima (MST).
Kruskal	Construção da árvore geradora mínima (MST).
BFS	Busca em largura (exploração por níveis).
DFS	Busca em profundidade (exploração recursiva do grafo).
Guloso	Definição de rota priorizando a relação entre demanda de alunos e distância.
TSP simplificado	Definição de uma sequência eficiente de visitas aos pontos.

Legenda: MST (*Minimum Spanning Tree*) corresponde à Árvore Geradora Mínima.
TSP (Caixeiro viajante simplificado)

Roteamento geográfico

- 1 A sequência de pontos calculada é enviada ao serviço OSRM.
- 2 O serviço retorna a geometria GeoJSON pelas ruas, as distâncias dos trechos, a distância total e a duração estimada.
- 3 As distâncias, recebidas em metros, são convertidas para quilômetros e apresentadas ao usuário.
- 4 A visualização ajusta automaticamente o zoom para enquadrar todo o percurso.
- 5 Sem acesso ao OSRM, o sistema utiliza geometria local e distância pela fórmula de Haversine.

Fallback de distância

Código 4: Pseudocódigo do cálculo de distância pela fórmula de Haversine.

```
1 FUNCAO calcularDistancia(lat1, lon1, lat2, lon2):  
2   // Formula de Haversine  
3   a = sin^2(deltaLat/2) +  
4       cos(lat1) * cos(lat2) * sin^2(deltaLon/2)  
5  
6   RETORNAR raioTerra * 2 * atan2(sqrt(a), sqrt(1-a))
```

Fonte: Adaptado de Maida [1].

Visualização e animação

- O desenho do percurso e a animação utilizam exatamente a mesma lista de coordenadas, preservando a correspondência visual.
- Um `javax.swing.Timer` atualiza o veículo aproximadamente a cada 16 ms.
- A posição é calculada pelo tempo transcorrido e pela velocidade configurada, com interpolação dentro do segmento atual.
- A orientação do veículo é obtida com `atan2`, de acordo com a direção do movimento.
- Coalescência e limitação de intervalos excessivos reduzem travamentos e saltos; a interface também oferece pausa e reinício.

Persistência, exportação e uso

- Projetos podem ser abertos e salvos para continuidade do trabalho.
- Resultados podem ser exportados nos formatos CSV, TXT e PDF.
- A interface apresenta a sequência de paradas, a distância de cada trecho, a quilometragem total e a duração estimada.
- A exibição do mapa e o roteamento viário completo requerem internet, pois utilizam dados do OpenStreetMap e o serviço OSRM.
- O fallback local mantém os cálculos básicos disponíveis sem o serviço externo de roteamento.

Diagrama de Classes Simplificado

Principais Classes do Sistema

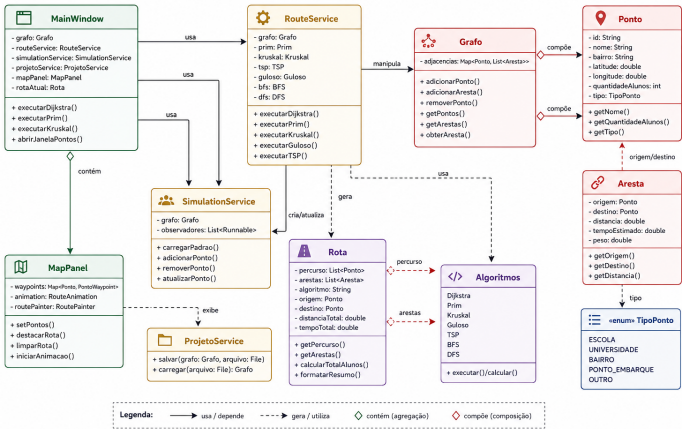


Figura 1: Diagrama simplificado das principais classes e seus relacionamentos.

Interface Principal

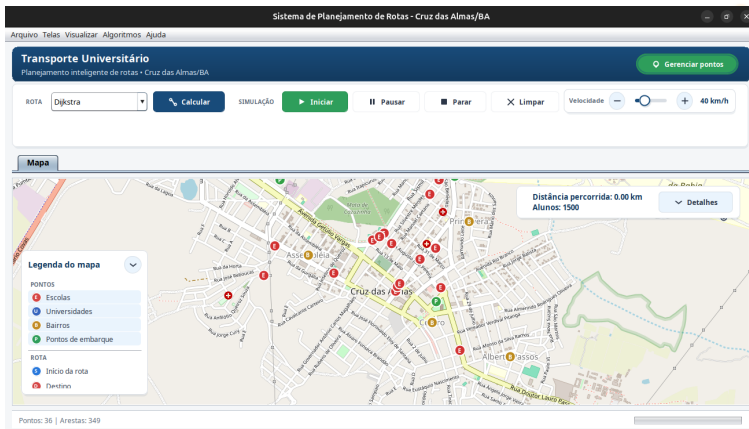


Figura 2: Interface principal do sistema de planejamento de rotas.

Fonte: Elaborado pelos autores.

Cadastro

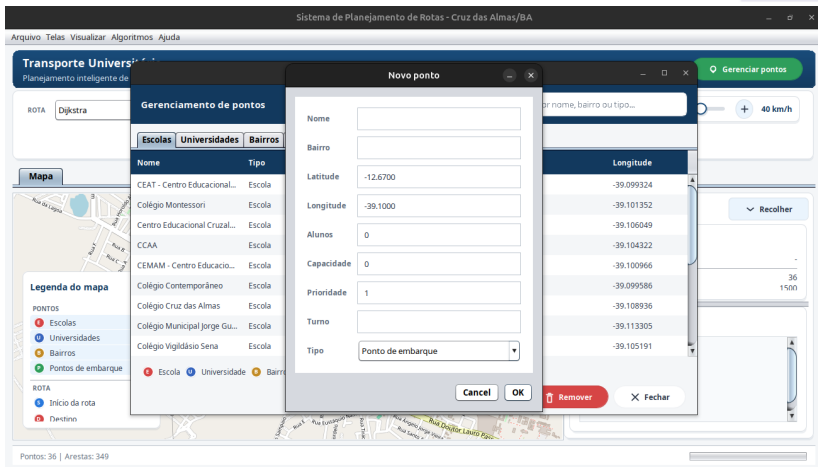


Figura 3: Cadastro de novos pontos.

Fonte: Elaborado pelos autores.

Execução do Algoritmo de Dijkstra

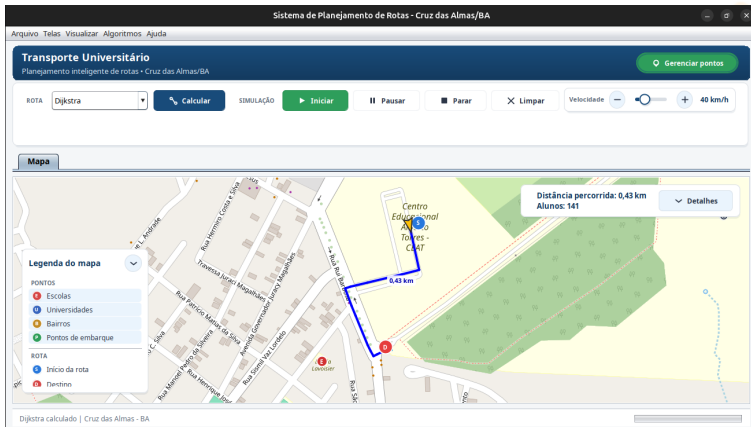


Figura 4: Menor caminho calculado pelo algoritmo de Dijkstra.

Fonte: Elaborado pelos autores.

Resumo da rota e estatísticas

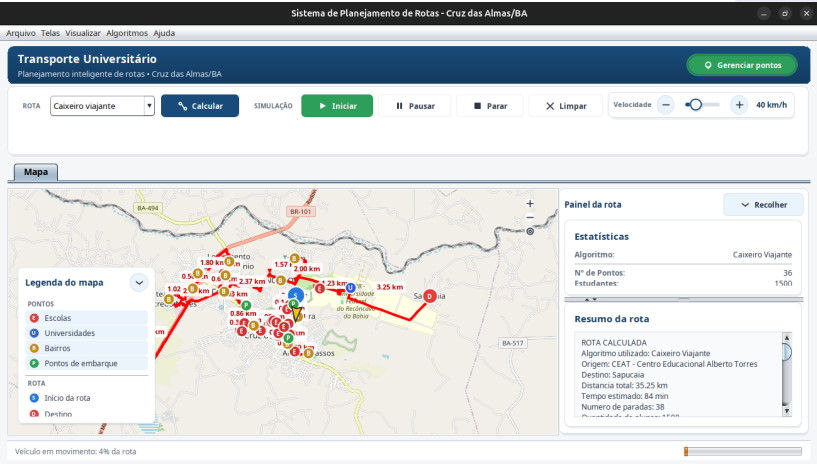


Figura 5: Resumo e estatísticas da rota calculada.

Resultados dos Algoritmos

Tabela 2: Métricas de percurso no grafo de 36 pontos, 347 arestas e 1500 alunos.

Algoritmo	Distância	Tempo	Alunos	Ponto final
Dijkstra	0,43 km	2 min	141	Portaria UFRB
Prim	50,57 km	131 min	1500	CEAT (retorno)
Kruskal	50,57 km	131 min	1500	CEAT (retorno)
BFS	76,16 km	180 min	1500	Embira
DFS	67,34 km	154 min	1500	Sapucaia
Guloso	52,89 km	125 min	1500	Sapucaia
TSP simplificado	35,25 km	84 min	1500	Sapucaia

Fonte: Elaborado pelos autores.

Principais resultados

- Prim e Kruskal produziram os mesmos resultados.
- O TSP simplificado apresentou a menor distância entre os algoritmos que visitam todos os pontos.

Principais conclusões

- Grafos são uma abstração eficaz para representar problemas reais de mobilidade urbana e logística.
- O sistema atingiu o objetivo de calcular e visualizar rotas mais eficientes.
- A solução contribui para reduzir distância e tempo estimados.
- A priorização por demanda mostrou-se útil para otimizar o atendimento aos pontos mais críticos.
- A aplicação consolidou o aprendizado de Dijkstra, Prim, Kruskal e heurística TSP.

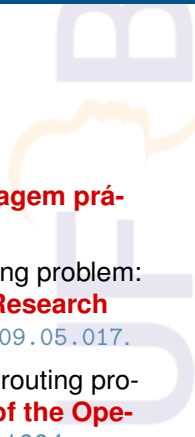
Limitações do estudo

- O TSP implementado é uma heurística gulosa, sem garantia de solução ótima.
- O VRP é simplificado, com número fixo de ônibus e capacidade definida.
- As distâncias via Haversine não refletem totalmente o traçado real das ruas nem o trânsito.
- Pontos e demandas foram inseridos manualmente, sem integração com dados reais de matrícula ou mobilidade.

Pesquisas futuras

- Integrar APIs de mapas reais, como Google Maps ou OSRM, para distâncias de rua e trânsito em tempo real.
- Adicionar rastreamento em tempo real dos ônibus e recálculo dinâmico de rotas.
- Explorar otimização multiobjetivo, considerando custo, tempo, conforto e demanda dos estudantes.
- Usar aprendizado de máquina para prever demanda por ponto e horário.
- Migrar a aplicação desktop para versão web ou mobile.

Referências I

- 
- [1] João Paulo Maida. **Teoria dos Grafos: Uma abordagem prática em Java**. Casa do Código, 2020.
 - [2] Junhyuk Park e Byung-In Kim. “The school bus routing problem: A review”. Em: **European Journal of Operational Research** 202.2 (2010), pp. 311–319. DOI: 10.1016/j.ejor.2009.05.017.
 - [3] Tolga Bektaş e Seda Elmastaş. “Solving school bus routing problems through integer programming”. Em: **Journal of the Operational Research Society** 58.12 (2007), pp. 1599–1604. DOI: 10.1057/palgrave.jors.2602305.



Obrigado(a) pela Atenção!